

Reviewer Pack — Secant Variety Dimension Lower-Bounds Disjointness Communica...

Entry 7181420017d3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 21:46:31 UTC

Secant Variety Dimension Lower-Bounds Disjointness Communication Complexity

Entry ID: 7181420017d3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 21:46:31 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry of Secant Varieties **Field B** (complexity object): Randomized Communication Complexity of Disjointness

Statement:

For a $|X| \times |Y|$ communication matrix M of the DISJOINTNESS function, the secant variety dimension of M 's rank-2 tensor decomposition satisfies $\dim(\sigma_2(M)) \geq c \cdot \min(|X|, |Y|)$ for some universal constant $c > 0$. This implies randomized communication complexity $R(M) \geq \Omega(\min(|X|, |Y|))$

Rationale (proposer's reasoning):

Secant variety dimensions capture the minimal number of rank-1 tensors needed to approximate a matrix, which directly relates to communication complexity via the tensor rank barrier. The DISJOINTNESS matrix's structure forces a linear lower bound on this dimension, revealing geometric constraints on information-theoretic protocols.

Taxonomy category: COMM_DISJ (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 43ee96ca1d11ced3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def lu_decomposition(A):
    n = len(A)
    L = [[0 for _ in range(n)] for _ in range(n)]
    U = [[0 for _ in range(n)] for _ in range(n)]

    for i in range(n):
        U[i][i] = A[i][i]
        for j in range(i+1, n):
            sum_upper = sum(L[j][k] * U[k][i] for k in range(i))
            U[j][i] = A[j][i] - sum_upper
        for j in range(i, n):
            if i == 0:
                L[j][i] = A[j][i]
            else:
                sum_lower = sum(L[j][k] * U[k][i] for k in range(i))
                L[j][i] = (A[j][i] - sum_lower) / U[i][i]

    return L, U

def tensor_rank(M):
    n = len(M)
    A = [[0 for _ in range(n)] for _ in range(n)]
    B = [[0 for _ in range(n)] for _ in range(n)]

    for i in range(n):
        for j in range(n):
            A[i][j] = M[2*i][2*j]
            B[i][j] = M[2*i+1][2*j+1]

    _, U_A = lu_decomposition(A)
    _, U_B = lu_decomposition(B)

    rank_A = sum(1 for row in U_A if any(val != 0 for val in row))
    rank_B = sum(1 for row in U_B if any(val != 0 for val in row))
```

```

    return max(rank_A, rank_B)

def generate_disjointness_matrix(n):
    X = set(range(n))
    Y = set(range(n))
    M = [[0 for _ in range(n)] for _ in range(n)]

    for i in range(n):
        for j in range(n):
            if (i in X and j in Y) or (i not in X and j not in Y):
                M[i][j] = 1

    return M

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    metric_values = []

    for n in n_values:
        M = generate_disjointness_matrix(n)
        rank = tensor_rank(M)
        metric_values.append(rank)

    if len(metric_values) < 30:
        return {
            "metric_name": "tensor_rank",
            "metric_value": sum(metric_values) / len(metric_values),
            "instances_tested": len(metric_values),
            "conjecture_holds": False,
            "counterexample": "not_enough_instances"
        }

    slope, intercept = linear_regression(n_values, metric_values)
    if slope < 0.9 * n_values[0]:
        return {
            "metric_name": "tensor_rank",
            "metric_value": sum(metric_values) / len(metric_values),
            "instances_tested": len(metric_values),
            "conjecture_holds": False,
            "counterexample": f"sublinear growth: slope={slope}"
        }

    return {
        "metric_name": "tensor_rank",
        "metric_value": sum(metric_values) / len(metric_values),
        "instances_tested": len(metric_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

def linear_regression(x, y):
    n = len(x)
    sum_x = sum(x)
    sum_y = sum(y)

```

```

sum_xy = sum(xi * yi for xi, yi in zip(x, y))
sum_xx = sum(xi ** 2 for xi in x)

slope = (n * sum_xy - sum_x * sum_y) / (n * sum_xx - sum_x ** 2)
intercept = (sum_y - slope * sum_x) / n

return slope, intercept

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]]
    if not seeds:
        seeds = [2**i + 3**j + 5**k for i in range(5) for j in range(5) for k in range(5)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")

    mean = sum(result["metric_value"] for result in results) / len(results)
    std = math.sqrt(sum((result["metric_value"] - mean) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"sublinear growth\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc52ee64.py", line 123, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc52ee64.py", line 73, in run_trial
    rank = tensor_rank(M)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc52ee64.py", line 44, in tensor_rank
    B[i][j] = M[2*i+1][2*j+1]
              ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing data, preventing evaluation of conjecture support/falsification | next: Fix tensor_rank implementation to handle matrix dimensions properly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	36190
2	preregistration	ollama_remote	qwen3:8b	0	0	19985
3	novelty	ollama_remote	qwen3:8b	0	0	16567
4	novelty	ollama_remote	qwen3:8b	0	0	10143
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12317
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14105
7	judge	ollama_remote	qwen3:8b	0	0	10902

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120210 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/7181420017d3.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7181420017d3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7181420017d3.tar.gz (if generated)